



# Programming Java

## Input and Output

Incheon Paik



## Contents

- Files and Directories
- Character Streams
- Buffered Character Streams
- The **PrintWriter** Class
- Byte Streams
- Random Access Files
- The **StreamTokenizer** Class
- Object Serialization
- The Java New I/O
- Writing Files
- Reading Files

## Files and Directories

The File class encapsulates information about the properties of a file or directory

### File Constructors

```
File(String path)
File(String directoryPath, String filename)
File(File directory, String filename)
```

### Methods Defined by the File class

```
boolean canRead();      boolean canWrite()
boolean delete();       boolean equals(Object obj)
boolean exists();       boolean exists()
String getAbsolutePath();
String getCanonicalPath() throws IOException
String getName();       String getParent()
String getPath();       boolean isAbsolute()
```

### Methods Defined by the File class

```
boolean isDirectory()
boolean isFile()
long lastModified()
long length()
String[] list()
boolean mkdir()
boolean mkdirs()
boolean renameTo(File newName)
```

<http://java.sun.com/j2se/1.4.2/docs/api/java/io/File.html>

3

Java

Computer Industry Lab.

## Files and Directories

```
class FileDemo {

    public static void main(String args[]) {

        try {

            // Display Constant
            System.out.println("pathSeparatorChar = " +
                File.pathSeparatorChar);
            System.out.println("separatorChar = " +
                File.separatorChar);

            // Test Some Methods
            File file = new File(args[0]);
            System.out.println("getName() = " +
                file.getName());
            System.out.println("getParent() = " +
                file.getParent());
            System.out.println("getAbsolutePath() = " +
                file.getAbsolutePath());
            System.out.println("getCanonicalPath() = " +
                file.getCanonicalPath());
            System.out.println("getPath() = " +
                file.getPath());

        }

    }

}
```

```
System.out.println("canRead() = " +
    file.canRead());
System.out.println("canWrite() = " +
    file.canWrite());
}
catch(Exception e) {
    e.printStackTrace();
}
}
```

### Result :

```
pathSeparatorChar = ;
separatorChar = \
getName() = FileDemo.java
getParent() = null
getAbsolutePath() = D:\lecture\2004-
01\teachyourself\example10-11\FileDemo.java
getCanonicalPath() = D:\lecture\2004-
01\teachyourself\example10-11\FileDemo.java
getPath() = FileDemo.java
canRead() = true
canWrite() = true
```

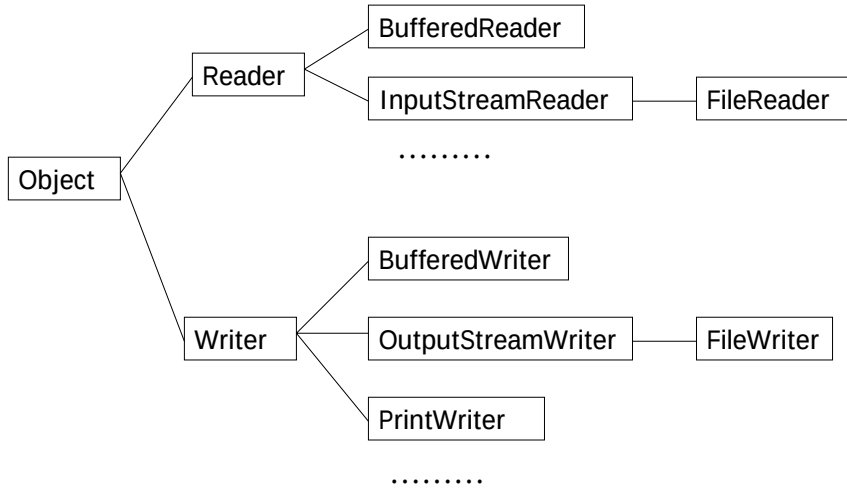
4

Java

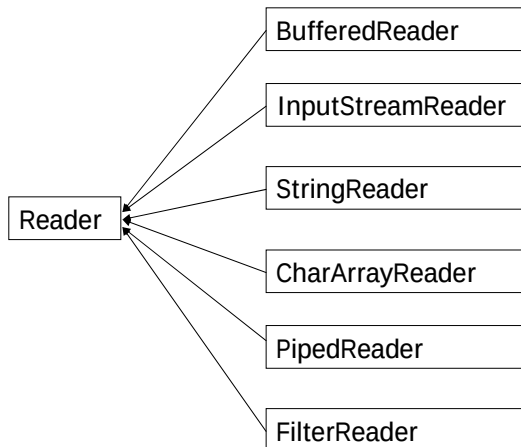
Computer Industry Lab.



## Character Streams

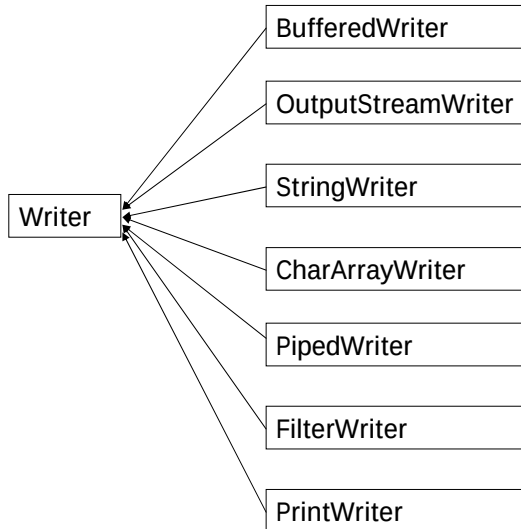


## Character Streams





## Character Streams



## Character Streams

### Writer Constructors

*Writer()*  
*Writer(Object obj)*

### OutputStreamWriter Constructors

*OutputStreamWriter(OutputStream os)*  
*OutputStreamWriter(OutputStream os, String encoding)*

### Methods Defined by the Writer

*abstract void close() throws IOException*  
*abstract void flush() throws IOException*  
*void write(int c) throws IOException*  
*void write(char buffer[]) throws IOException*  
*abstract void write(char buffer[], int index, int size) throws IOException*  
*void write(String s) throws IOException*  
*void write(String s, int index, int size) throws IOException*

### getEncoding() Method

*String getEncoding()*

### FileWriter Constructors

*FileWriter(String filepath) throws IOException*  
*FileWriter(String filepath, boolean append) throws IOException*  
*FileWriter(String filepath) throws IOException*

Refer to <http://java.sun.com/j2se/1.5.0/docs/api/java/io/Writer.html>



## Character Streams

### Methods Defined by the Reader

```
abstract void close() throws IOException
void mark(int numChars) throws IOException
boolean markSupported()
int read() throws IOException
int read(char buffer[]) throws IOException
int read(char buffer[], int offset, int numChars) throws IOException
boolean ready() throws IOException
void reset() throws IOException
int skip(long numChars) throws IOException
```

### InputStreamWriter Constructors

```
InputStreamWriter(InputStream os)
InputStreamWriter(InputStream os, String encoding)
```

### getEncoding() Method

```
String getEncoding()
```

### FileReader Constructors

```
FileReader(String filepath) throws FileNotFoundException
FileReader(File fileObj) throws FileNotFoundException
```

Refer to <http://java.sun.com/j2se/1.5.0/docs/api/java/io/Reader.html>



## Character Stream Examples

```
import java.io.*;
class FileWriterDemo {
    public static void main(String args[]) {
        try {
            // Create a FileWriter
            FileWriter fw = new FileWriter(args[0]);
            // Write string to the file
            for (int i = 0; i < 12; i++) {
                fw.write("Line " + i + "\n");
            }
            // Close a FileWriter
            fw.close();
        } catch (Exception e) {
            System.out.println("Exception: " + e);
        }
    }
}
```

Run :

```
java FileWriterDemo output.txt
java FileReaderDemo output.txt
```

```
class FileReaderDemo {
    public static void main(String args[]) {
        try {
            FileReader fr = new FileReader(args[0]);
            int i;
            while((i = fr.read()) != -1) {
                System.out.print((char)i);
            }
            fr.close();
        } catch (Exception e) {
            System.out.println("Exception: " + e);
        }
    }
}
```

Result :

```
Line 0
Line 1
Line 2
Line 3
Line 4
Line 5
Line 6
Line 7
Line 8
Line 9
Line 10
Line 11
```

## Buffered Character Streams

### BufferedWriter Constructors

```
BufferedWriter(Writer w)
BufferedWriter(Writer w, int bufSize)
```

### newLine() Method

```
void newLine() throws IOException
```

### BufferedReader Constructors

```
BufferedReader(Reader r)
BufferedReader(Reader r, int bufSize)
```

### readLine() Method

```
String readLine() throws IOException
```

Refer to  
<http://java.sun.com/j2se/1.5.0/docs/api/java/io/BufferedWriter.html>  
<http://java.sun.com/j2se/1.5.0/docs/api/java/io/BufferedReader.html>

```
class BufferedWriterDemo {
    public static void main(String args[]) {
        try {
            FileWriter fw = new FileWriter(args[0]);
            BufferedWriter bw = new BufferedWriter(fw);
            for (int i = 0; i < 12; i++) {
                bw.write("Line " + i + "\n");
            }
            bw.close();
        } catch (Exception e) {
            System.out.println("Exception: " + e);
        }
    }
}
```

Java

11

Computer Industry Lab.

## Character Stream Examples

```
class BufferedReaderDemo {
    public static void main(String args[]) {
        try {
            FileReader fr = new FileReader(args[0]);
            BufferedReader br = new BufferedReader(fr);
            String s;
            while((s = br.readLine()) != null)
                System.out.println(s);
            fr.close();
        } catch (Exception e) {
            System.out.println("Exception: " + e);
        }
    }
}
```

Run :

```
java BufferedWriterDemo output.txt
java BufferedReaderDemo output.txt
```

```
class ReadConsole {
    public static void main(String args[]) {
        try {
            InputStreamReader isr =
                new InputStreamReader(System.in);
            BufferedReader br = new BufferedReader(isr);
            String s;
            while((s = br.readLine()) != null) {
                System.out.println(s.length());
            }
            isr.close();
        } catch (Exception e) {
            System.out.println("Exception: " + e);
        }
    }
}
```

Result :

```
Line 0
Line 1
Line 2
Line 3
Line 4
Line 5
Line 6
Line 7
Line 8
Line 9
Line 10
Line 11
```

Java

12

Computer Industry Lab.



## The PrintWriter Class

### PrintWriter Constructor

```
PrintWriter(OutputStream outputStream)
PrintWriter(OutputStream outputStream, boolean flushOnNewline)
PrintWriter(Writer writer)
PrintWriter(Writer writer, boolean flushOnNewline)
```

#### Refer to

<http://java.sun.com/j2se/1.4.2/docs/api/java/io/PrintWriter.html>

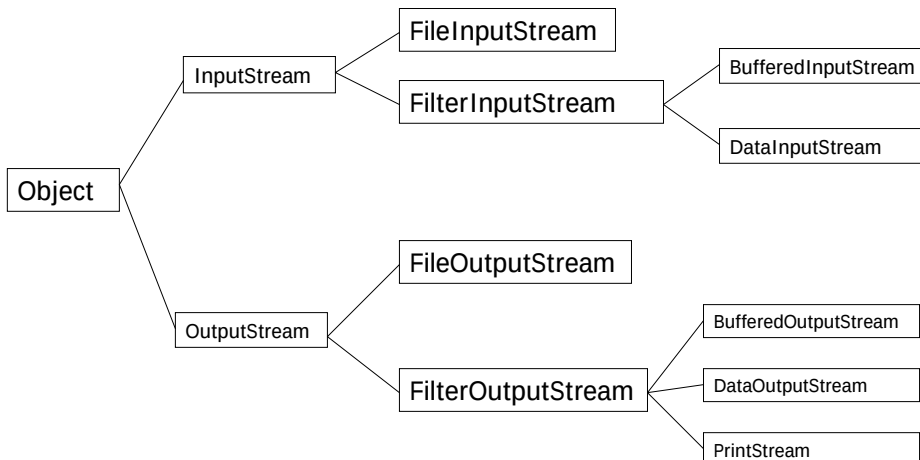
#### Result:

```
true
A
500
40000
45.67
45.67
Hello
99
```

```
class PrintWriterDemo {
    public static void main(String args[]) {
        try {
            PrintWriter pw = new PrintWriter(System.out);
            pw.println(true);
            pw.println('A');
            pw.println(500);
            pw.println(40000L);
            pw.println(45.67f);
            pw.println(45.67);
            pw.println("Hello");
            pw.println(new Integer("99"));
            pw.close();
        }
        catch (Exception e) {
            System.out.println("Exception: " + e);
        }
    }
}
```

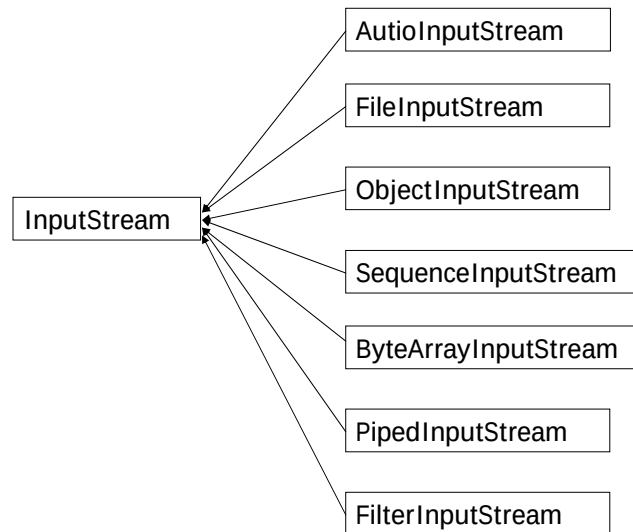


## Byte Streams (Binary Streams)

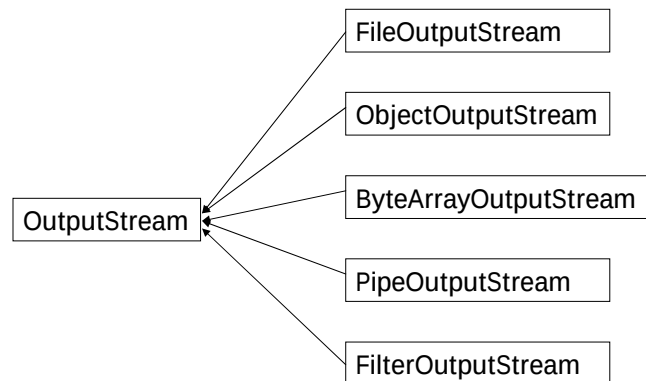




## Byte Streams



## Byte Streams







## Byte Streams

### Methods Defined by the OutputStream

```
void close() throws IOException
void flush() throws IOException
void write(int i) throws IOException
void write(byte buffer[]) throws IOException
void write(char buffer[], int index, int size) throws IOException
```

### FileOutputStream Constructor

```
FileOutputStreamWriter(String filepath) throws IOException
FileOutputStreamWriter(String filepath, boolean append) throws IOException
FileOutputStreamWriter(File fileObj) throws IOException
```

### FilterOutputStream Constructor

```
FilterOutputStream(OutputStream os)
```

### BufferedOutputStream Constructor

```
BufferedOutputStream(OutputStream os)
BufferedOutputStream(OutputStream os, int bufSize)
```

### DataOutputStream Constructor

```
DataOutputStream(OutputStream os)
```

Refer to <http://java.sun.com/j2se/1.4.2/docs/api/java/io/OutputStream.html>



## Byte Streams (DataOutput Interface)

### Methods Defined by the DataOutput

```
void write(int i) throws IOException
void write(byte buffer[]) throws IOException
void write(byte buffer[], int index, int size) throws IOException
void writeBoolean(boolean b) throws IOException
void writeByte(int i) throws IOException
void writeByte(String s) throws IOException
void writeChar(int i) throws IOException
void writeChars(String s) throws IOException
void writeDouble(double d) throws IOException
void writeFloat(float f) throws IOException
void writeInt(int i) throws IOException
void writeLong(long l) throws IOException
void writeShort(short s) throws IOException
void writeUTF(String s) throws IOException
```

### PrintStream Constructors

```
PrintStream(OutputStream outputStream)
PrintStream(OutputStream outputStream, boolean flushOnNewline)
```

Refer to <http://java.sun.com/j2se/1.5.0/docs/api/java/io/DataOutput.html>



## Byte Streams (InputStream Interface)

### Methods Defined by the InputStream

*int available()* throws *IOException*  
*void close()* throws *IOException*  
*void mark(int numBytes)*  
*boolean markSupported()*  
*int read()* throws *IOException*  
*int read(byte buffer[])* throws *IOException*  
*int read(byte buffer[], int offset, int numBytes)* throws *IOException*  
*void reset()* throws *IOException*  
*int skip(long numBytes)* throws *IOException*

### FileInputStream Constructors

*FileInputStream(String filepath)* throws *FileNotFoundException*  
*FileInputStream(File fileObj)* throws *FileNotFoundException*

### FilterInputStream Constructor

*FilterInputStream(InputStream is)*

### BufferedInputStream Constructors

*BufferedInputStream(InputStream is)*  
*BufferedInputStream(InputStream is, int bufSize)*

### DataInputStream Constructor

*DataStream(InputStream is)*

Refer to <http://java.sun.com/j2se/1.5.0/docs/api/java/io/InputStream.html>



## Byte Streams (DataInput Interface)

### Methods Defined by DataInput

*boolean readBoolean()* throws *IOException*  
*byte readByte()* throws *IOException*  
*char readChar()* throws *IOException*  
*double readDouble()* throws *IOException*  
*float readFloat()* throws *IOException*  
*void readFully(byte buffer[])* throws *IOException*  
*void readFully(byte buffer[], int index, int size)* throws *IOException*  
*int readInt()* throws *IOException*  
*String readLine()* throws *IOException*  
*long readLong()* throws *IOException*  
*short readShort()* throws *IOException*  
*String readUTF()* throws *IOException*  
*int readUnsignedByte()* throws *IOException*  
*int readUnsignedShort()* throws *IOException*  
*int skipBytes(int n)* throws *IOException*

```
class FileOutputStreamDemo {
    public static void main(String args[]) {
        try {
            FileOutputStream fos =
                new FileOutputStream(args[0]);
            for (int i = 0; i < 12; i++) {
                fos.write(i);
            }
            fos.close();
        }
        catch (Exception e) {
            System.out.println("Exception: " + e);
        }
    }
}
```

Refer to <http://java.sun.com/j2se/1.5.0/docs/api/java/io/DataInput.html>



## Byte Streams (Example)

```
class FileInputStreamDemo {  
  
    public static void main(String args[]) {  
  
        try {  
  
            // Create FileInputStream  
            FileInputStream fis =  
                new FileInputStream(args[0]);  
  
            // Read and Display data  
            int i;  
            while ((i = fis.read()) != -1) {  
                System.out.println(i);  
            }  
  
            // Close FileInputStream  
            fis.close();  
        }  
        catch (Exception e) {  
            System.out.println("Exception: " + e);  
        }  
    }  
}
```

Run :

```
java FileOutputStreamDemo output.txt  
java FileInputStreamDemo output.txt
```

Result :

```
0  
1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
11
```



## Buffered[Input/Output]Stream Examples

```
import java.io.*;  
class BufferedOutputStreamDemo {  
    public static void main(String args[]) {  
        try {  
            FileOutputStream fos =  
                new FileOutputStream(args[0]);  
            BufferedOutputStream bos =  
                new BufferedOutputStream(fos);  
            for (int i = 0; i < 12; i++) {  
                bos.write(i);  
            }  
            bos.close();  
        }  
        catch (Exception e) {  
            System.out.println("Exception: " + e);  
        }  
    }  
}
```

Run :

```
java BufferedOutputStreamDemo output.txt  
java BufferedInputStreamDemo output.txt
```

```
class BufferedInputStreamDemo {  
    public static void main(String args[]) {  
        try {  
            FileInputStream fis =  
                new FileInputStream(args[0]);  
            BufferedInputStream bis =  
                new BufferedInputStream(fis);  
            int i;  
            while((i = bis.read()) != -1) {  
                System.out.println(i);  
            }  
            bis.close();  
        }  
        catch (Exception e) {  
            System.out.println("Exception: " + e);  
        }  
    }  
}
```

## DataOutputStream Examples

```
class DataOutputStreamDemo {
    public static void main(String args[]) {
        try {
            FileOutputStream fos =
                new FileOutputStream(args[0]);
            DataOutputStream dos =
                new DataOutputStream(fos);
            dos.writeBoolean(false);
            dos.writeByte(Byte.MAX_VALUE);
            dos.writeChar('A');
            dos.writeDouble(Double.MAX_VALUE);
            dos.writeFloat(Float.MAX_VALUE);
            dos.writeInt(Integer.MAX_VALUE);
            dos.writeLong(Long.MAX_VALUE);
            dos.writeShort(Short.MAX_VALUE);
            fos.close();
        }
        catch (Exception e) {
            System.out.println("Exception: " + e);
        }
    }
}
```

Run :

```
java DataOutputStreamDemo output.txt
java DataInputStreamDemo output.txt
```

```
class DataInputStreamDemo {
    public static void main(String args[]) {
        try {
            FileInputStream fis =
                new FileInputStream(args[0]);
            DataInputStream dis =
                new DataInputStream(fis);
            System.out.println(dis.readBoolean());
            System.out.println(dis.readByte());
            System.out.println(dis.readChar());
            System.out.println(dis.readDouble());
            System.out.println(dis.readFloat());
            System.out.println(dis.readInt());
            System.out.println(dis.readLong());
            System.out.println(dis.readShort());
            fis.close();
        }
        catch (Exception e) {
            System.out.println("Exception: " + e);
        }
    }
}
```

Java

23

Computer Industry Lab.

## Random Access Files

### Methods Defined by the RandomAccessFile

*void close() throws IOException*  
*long getFilePointer() throws IOException*  
*long length() throws IOException*  
*int read() throws IOException*  
*int read(byte buffer[], int index, int size) throws IOException*  
*int read(byte buffer[]) throws IOException*  
*void seek(long n) throws IOException*  
*int skipBytes(int n) throws IOException*

Refer to <http://java.sun.com/j2se/1.5.0/docs/api/java/io/RandomAccessFile.html>

```
class Tail {
    public static void main(String args[]) {
        try {
            RandomAccessFile raf =
                new RandomAccessFile(args[0], "r");
            long count = Long.valueOf(args[1]).longValue();
```

```
            long position = raf.length();
            position -= count;
            if (position < 0)
                position = 0;
            raf.seek(position);

            while(true) {
                try {
                    byte b = raf.readByte();
                    System.out.print((char)b);
                }
                catch (EOFException eofe) {
                    break;
                }
            }
            catch (Exception e) {
                e.printStackTrace();
            }
        }
    }
}
```

Run :

```
java Tail Tail.java 40
```

Result :

```
e.printStackTrace();
}
```

Java

24

Computer Industry Lab.



## The StreamTokenizer Class

### StreamTokenizer Constructor

```
StreamTokenizer(Reader r)
```

### Methods Defined by the StreamTokenizer

```
void commentChar (int ch)
void eolIsSignificant(boolean flag)
int lineno()
void lowerCaseMode(boolean flag)
int nextToken() throws IOException
void ordinaryChar(int ch)
void parseNumbers()
void pushBack()
void quoteChar(int ch)
void resetSyntax()
```

Refer <http://java.sun.com/j2se/1.5.0/docs/api/java/io/StreamTokenizer.html>

### Methods Defined by the StreamTokenizer

```
void slashSlashComments(boolean flag)
String toString()
void whitespaceChars(int c1, int c2)
void wordChars(int c1, int c2)
```

### General Procedure to use a StreamTokenizer

1. Create a StreamTokenizer object for a Reader.
2. Define how characters are to be processed.
3. Call nextToken() to obtain the next token.
4. Read the ttype instance variable to determine the token type.
5. Read the value of the token from the sval, nval, or ttype instance variable.
6. Process the token.
7. Repeat steps 3-6 until nextToken() returns StreamTokenizer.TT\_EOF.



## StreamTokenizer Example 1

```
class StreamTokenizerDemo {
    public static void main(String args[]) {
        try {
            // Create FileReader
            FileReader fr = new FileReader(args[0]);

            // Create BufferedReader
            BufferedReader br = new BufferedReader(fr);

            // Create StreamTokenizer
            StreamTokenizer st = new StreamTokenizer(br);

            // Define period as ordinary character
            st.ordinaryChar('.');

            // Define apostrophe as word character
            st.wordChars('\'', '');
```

Tokens.txt :

The price is \$23.45.  
Is that too expensive?  
(I don't think so.)

```
//Process tokens
while(st.nextToken() != StreamTokenizer.TT_EOF)
{
    switch(st.ttype) {
        case StreamTokenizer.TT_WORD:
            System.out.println(st.lineno() + ") " +
                st.sval);
            break;
        case StreamTokenizer.TT_NUMBER:
            System.out.println(st.lineno() + ") " +
                st.nval);
            break;
        default:
            System.out.println(st.lineno() + ") " +
                (char)st.ttype);
    }
}
fr.close();
}
catch (Exception e) {
    System.out.println("Exception: " + e);
}
}
```



## StreamTokenizer Example 2

```
class StreamTokenizerDemo2 {  
    public static void main(String args[]) {  
        try {  
            // Create FileReader  
            FileReader fr = new FileReader(args[0]);  
            // Create BufferedReader  
            BufferedReader br = new BufferedReader(fr);  
            // Create StreamTokenizer  
            StreamTokenizer st = new StreamTokenizer(br);  
            // Consider commas as white space  
            st.whitespaceChars(',', ' ');  
        }  
    }  
}
```

```
numbers.txt :  
34.567, 23, -9.3  
21, -23, 90, 7.6
```

```
//Process tokens  
while(st.nextToken() != StreamTokenizer.TT_EOF)  
{  
    switch(st.ttype) {  
        case StreamTokenizer.TT_WORD:  
            System.out.println(st.lineno() + ") " +  
                st.sval);  
            break;  
        case StreamTokenizer.TT_NUMBER:  
            System.out.println(st.lineno() + ") " +  
                st.nval);  
            break;  
        default:  
            System.out.println(st.lineno() + ") " +  
                (char)st.ttype);  
    }  
}  
fr.close();  
}  
catch (Exception e) {  
    System.out.println("Exception: " + e);  
}  
}
```



## Object Serialization

### What is Object Serialization?

- **Process of reading and writing objects**
  - Writing an object is to represent its state in a serialized form sufficient to reconstruct the object as it is read.
  - Object serialization is essential to building all but the most transient applications.
- **Examples of using the object serialization**
  - Remote Method Invocation (RMI)--communication between objects via sockets
  - Lightweight persistence--the archival of an object for use in a later invocation of the same program



## Serializing Objects

### ■ How to Write to an ObjectOutputStream

- Writing objects to a stream is a straight-forward process. Example of constructing a Date object and then serializing that object:

```
FileOutputStream out = new FileOutputStream("theTime");  
ObjectOutputStream s = new ObjectOutputStream(out);  
s.writeObject("Today");  
s.writeObject(new Date());  
s.flush();
```



## Serializing Objects

### ■ How to Read from an ObjectOutputStream

- Example that reads in the String and the Date object that was written to the file named theTime in the read example:

```
FileInputStream in = new FileInputStream("theTime");  
ObjectInputStream s = new ObjectInputStream(in);  
String today = (String)s.readObject();  
Date date = (Date)s.readObject();
```



## Serializing Objects

### ■ Providing Object Serialization for Your Classes

- Implementing the Serializable Interface
- Customizing Serialization
- Implementing the Externalizable Interface
- Protecting Sensitive Information

[ObjectFileTest.java]

/home/course/prog3/examples/objserial/ObjectFileTest.java



## The Java New I/O

### The Java New I/O

The new I/O (NIO) APIs introduced in v 1.4 provide new features and improved performance in the areas of buffer management, scalable network and file I/O, character-set support, and regular-expression matching. The NIO APIs supplement the I/O facilities in the java.io package.

#### Features

- Buffers for data of primitive types
- Character-set encoders and decoders
- A pattern-matching facility based on Perl-style regular expressions
- Channels, a new primitive I/O abstraction
- A file interface that supports locks and memory mapping

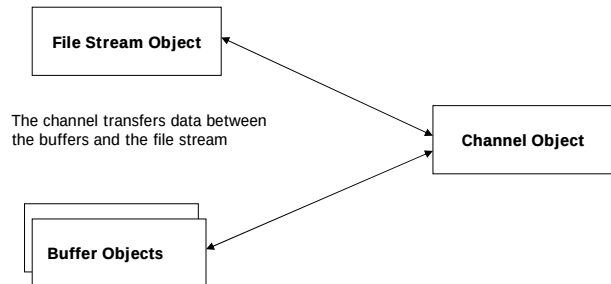




## The Java New File I/O

### For the New File I/O : Three Kinds of Objects are Involved

- A file stream object : FileOutputStream objects, FileInputStream objects
- One or more buffer objects : ByteBuffer, CharBuffer, LongBuffer, etc
- A channel object : FileChannel,...



## Writing Files

### Channels

Channels were introduced in the 1.4 release of Java to provide a faster capability for a faster capability for input and output operations with files, network sockets, and piped I/O operations between programs than the methods provided by the stream classes.

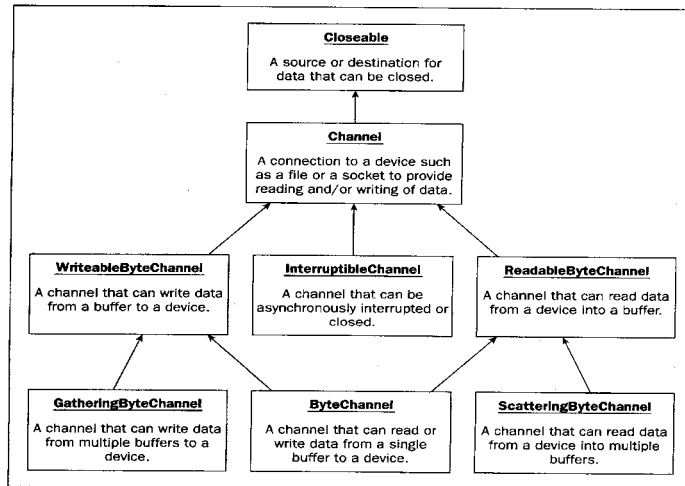
The channel mechanism can take advantage of buffering and other capabilities of the underlying operating system and therefore is considerably more efficient than using the operations provided directly within the file stream classes.

### A summary of the essential role of each of them in file operations

- A File object encapsulates a path to a file or a directory, and such an object encapsulating a file path can be used to construct a file stream object.
- A FileInputStream object encapsulates a file that can be read by a channel. A FileOutputStream object encapsulates a file that can be written by a channel.
- A buffer just holds data in memory. The loaded data to be written to a file will be saved at buffer using the buffer's put() method, and retrieved using buffer's get() methods.
- A FileChannel object can be obtained from a file stream object or a RandomAccessFile object.

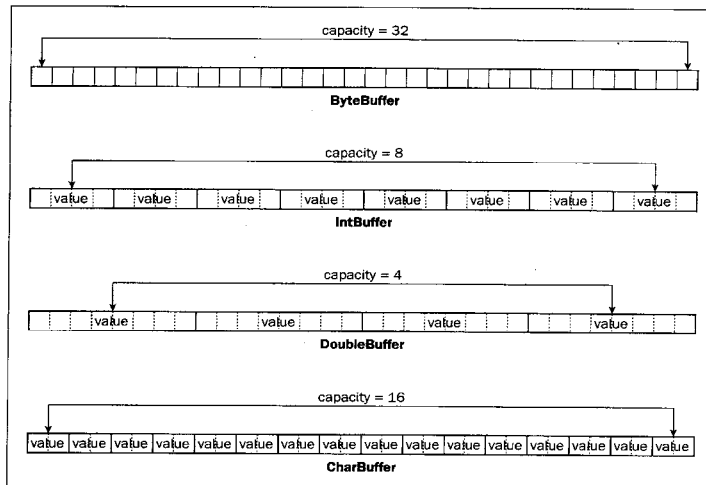
## Writing Files

### The hierarchy of the channel interfaces



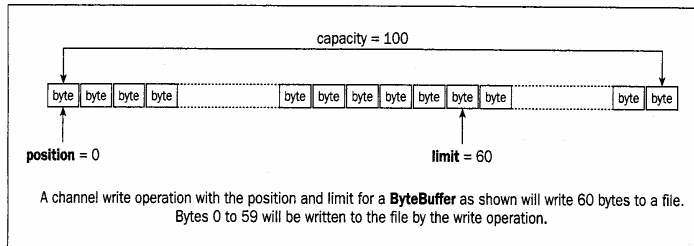
## Writing Files

### The Capacities of Different Buffers



## Writing Files

### An illustration of an operation that writes data from the buffer to a file



## Writing Files

### Methods for Setting the Position and Limit

*position(int newPosition) : Sets the position to the index value specified by the argument*

*limit(int newLimit) : Set the limit to the index value specified by the argument*

### Creating Buffers

```
ByteBuffer buf = ByteBuffer.allocate(1024);  
FloatBuffer floatBuf = FloatBuffer.allocate(100);
```

### View Buffers

```
ByteBuffer buf = ByteBuffer.allocate(1024);  
IntBuffer intBuf = buf.asIntBuffer();
```

### Methods for creating view buffers for a byte buffer object

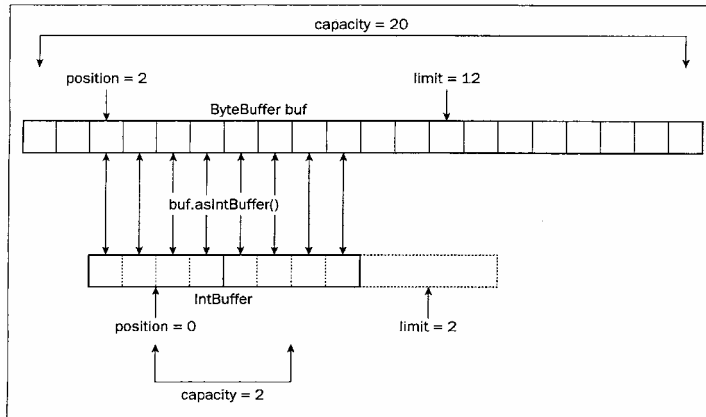
```
asCharBuffer()  
asShortBuffer()  
asIntBuffer()  
asLongBuffer()  
asFloatBuffer()  
asDoubleBuffer()  
asReadOnlyBuffer()
```

### clear(), flip(), and rewind() methods

*clear() : limit -> capacity, position -> 0*  
*flip() : limit -> current position, position -> 0*  
*rewind() : limit -> unchanged, position -> 0*

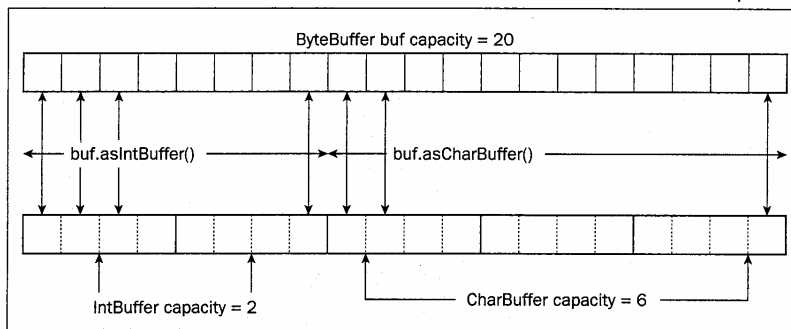
## Writing Files

An illustration of a view buffer of type `IntBuffer` that is created after the initial position of the byte buffer has been incremented by 2.



## Writing Files

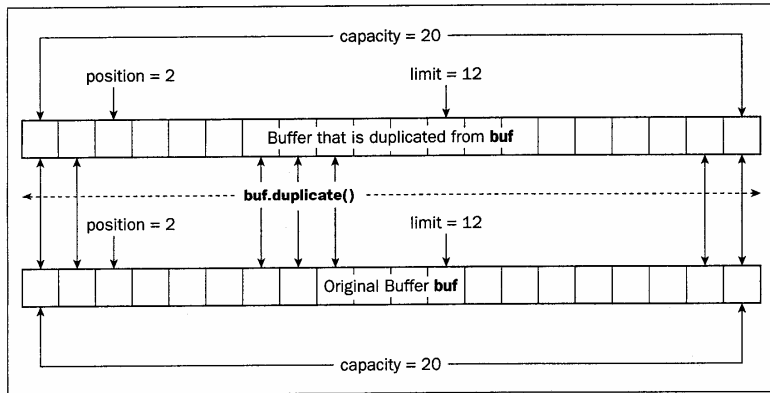
An illustration of mapping several different view buffers to a single byte buffer so that each provides a view of a different segment of the byte buffer in terms of a particular type of value.





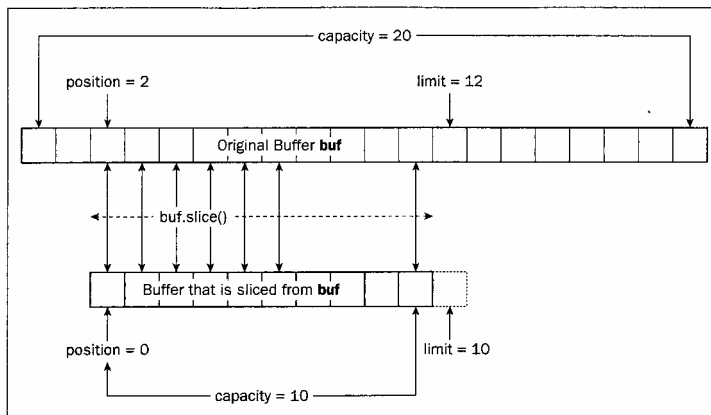
## Writing Files

### Duplicating Buffers



## Writing Files

### Slicing Buffers

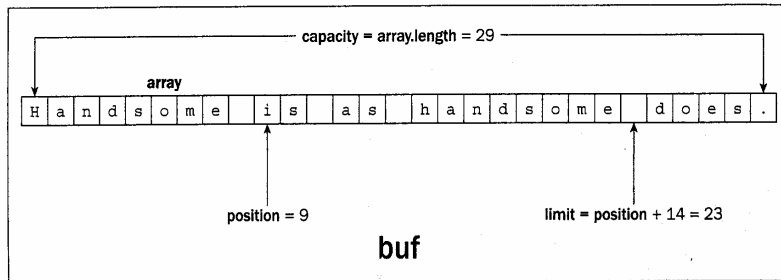




## Writing Files

### Creating Buffers by Wrapping Arrays

```
String saying = "Handsome is as handsome does.";
Byte[] array = saying.getBytes();
ByteBuffer buf = ByteBuffer.wrap(array, 9, 14);
```



## Writing Files

### Marking a Buffer

```
buf.mark(); // Mark the current position
buf.limit(519).position(256).mark();
buf.reset(); // Reset position to last marked
```

### Transferring Data into a Buffer

#### Methods of the ByteBuffer class

```
put (byte b)
put (int index, byte b)
put (byte[] array)
put (byte[] array, int offset, int length)
put (ByteBuffer src)
```

```
putDouble(double value)
putDouble(int index, double value)
```

### Using View Buffers

```
String text = "Value of e";
ByteBuffer buf = ByteBuffer.allocate(50);
CharBuffer charBuf = buf.asCharBuffer();
charBuf.put(text);
```

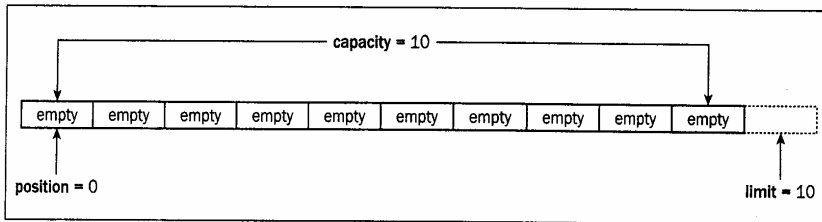
```
// Update byte buffer position by the number of bytes
// we have transferred
buf.position(buf.position() + 2*charBuf.position());
buf.putDouble(Math.E);
```



## Writing Files

### Preparing a Buffer for Output to a File

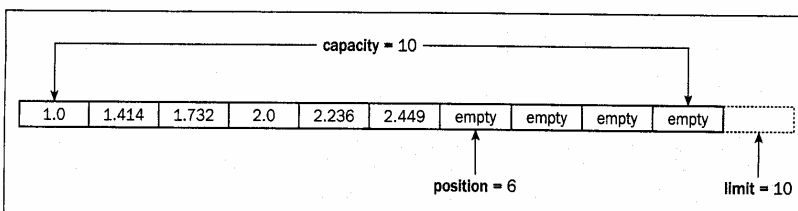
```
ByteBuffer buf = ByteBuffer.allocate(80);  
DoubleBuffer doubleBuf = buf.asDoubleBuffer();
```



## Writing Files

### Preparing a Buffer for Output to a File

```
double[] data = {1.0, 1.414, 1.732, 2.0, 2.236, 2.449};  
doubleBuf.put(data);
```

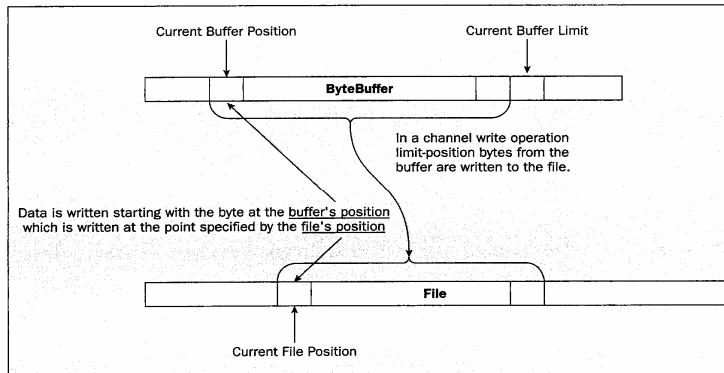


## Writing Files

### Writing to a File

```
try {  
    outputChannel.write(buf);  
} catch (IOException e) { }
```

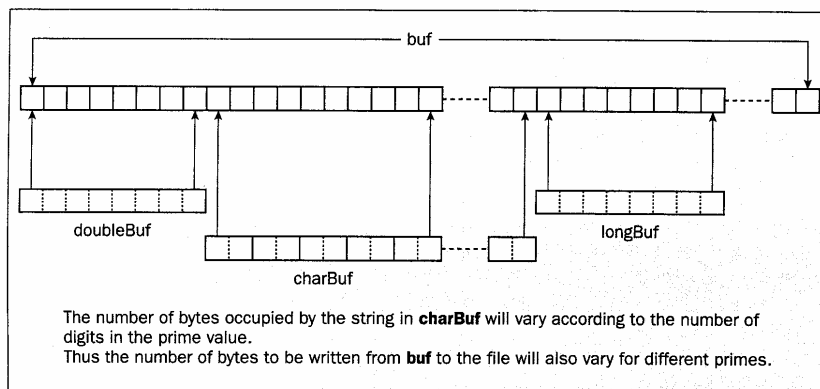
### File Position



## Writing Files

### Writing Mixed Data to a File

1. A count of the length of the string as binary value
2. The string representation of the prime value "prime = nnn", where obviously the number of digits will vary
3. The prime as a binary value of type long



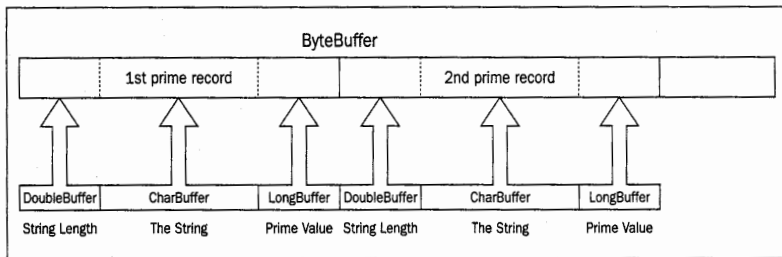




## Writing Files

### Multiple Records in a Buffer

Can load the byte buffer using three different view buffers repeatedly to put data for as many primes into the buffer as we can.



## Writing Files

### An Example (Writing Mixed Data)

#### An Example Program:

/home/course/prog3/java2-1.5/Code/Ch10/PrimesToFile2.java



## Reading Files

### Creating File Input Streams

```
File aFile = new File("charData.txt");
FileInputStream inFile = null;
try {
    inFile = new FileInputStream(aFile);
} catch (FileNotFoundException e) {
    e.printStackTrace(System.err);
    System.exit(1);
}
```

### File Channel Read Operations

```
FileChannel inChannel = inFile.getChannel();
ByteBuffer buf = ByteBuffer.allocate(48);
try {
    while(inChannel.read(buf) != -1) {
        System.out.println("String read: " + ((ByteBuffer)(buf.flip())).asCharBuffer().toString());
        buf.clear();
        // Clear the buffer for the next read
    }
    System.out.println("EOF reached.");
    inFile.close();
    // Close the file and the channel
} catch (IOException e) {
    e.printStackTrace(System.err);
    System.exit(1);
}
System.exit(0);
}
```

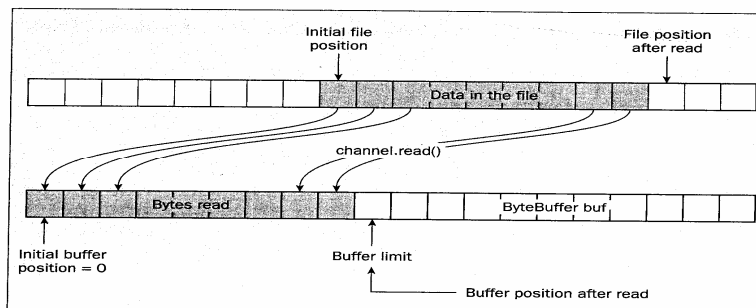


## Reading Files

### Three read() methods for a FileChannel object

```
read(ByteBuffer buf)
read(ByteBuffer[] buffers)
read(ByteBuffer[] buffers, int offset, int length)
```

### An Illustration of read operation (amount of data, the position and limit for the buffer)





## Reading Files

### An Example (Reading Mixed Data)

#### An Example Program:

`/home/course/prog3/java2-1.5/Code/Ch10/ReadPrimesMixedData.java`



## Exercise

### **Step 1 Creating Information Summarizer (Use the `FileInputStream` and `BufferedReader` Class)**

Related Slides : #5 – 10

### ■ **Step 2 (Using the `DataInput/OutputStream` Class)**

Related Slides : #14 – 19

### ■ **Step 3 (Re-write the Step 2 Using the New I/O package)**

Related Slides : #32 – 51

### ■ **Step 4 (Mixed-Data Processing Using the New I/O package)**

Related Slides : #32 – 52

### ■ **Step 5 (A Stream Tokenizer)**

Related Slides : #25-27

### ■ **Option (Object Serialization)**

Related Slides : #28-31